

Takeya-Inspired Polynomial Partitioning for Load Balancing in GPU Ray Tracing

Daniel Lougen
Gestalt Labs

May 20, 2026

Abstract

Modern GPU ray tracing suffers from severe load imbalance due to the highly variable computational cost of tracing rays through complex scenes. Rays hitting reflective or refractive surfaces may require 10-50 bounces, while rays hitting the sky terminate after 1 bounce, creating workload ratios exceeding 50:1 across pixels. Traditional tile-based rendering assigns fixed screen regions to GPU warps, causing the entire GPU to stall while waiting for the slowest warp. We present a novel approach inspired by the **Takeya conjecture** from harmonic analysis, which studies the geometric properties of line segments in different directions. Our method uses **polynomial partitioning**—a technique from algebraic geometry—to divide the image into cells with balanced total workload rather than balanced pixel counts. By constructing a polynomial whose zero-set separates high-workload regions (reflective objects) from low-workload regions (sky), we achieve load imbalance ratios of 1.03-1.09x compared to 1.62-1.73x for uniform tiling. This translates to GPU utilization improvements from 58-62% to 91-97%, with partition overhead of only 3-54ms amortized over render times of 0.4-6.9 seconds. Our approach is directly applicable to NVIDIA’s RT cores and could enable significant performance improvements in real-time ray tracing applications.

1 Introduction

Ray tracing has become the gold standard for photorealistic rendering, with NVIDIA’s RTX GPUs bringing real-time ray tracing to consumer hardware through dedicated RT cores. However, achieving high GPU utilization in ray tracing remains challenging due to the inherent variability in ray complexity.

1.1 The Load Imbalance Problem

In a typical ray-traced scene, the computational cost per ray varies dramatically:

- **Sky rays:** Terminate immediately, $\sim 50\text{ns}$ per ray
- **Diffuse surfaces:** 1-3 bounces, $\sim 150\text{-}450\text{ns}$ per ray
- **Reflective surfaces:** 5-10 bounces, $\sim 500\text{-}1000\text{ns}$ per ray
- **Glass/volumetrics:** 10-50 bounces with refraction, $\sim 1500\text{-}2500\text{ns}$ per ray

When using traditional uniform tiling (e.g., 16×16 pixel tiles), a tile containing a reflective sphere may take 10-50 $\times$ longer to render than a tile containing only sky. Since GPU warps execute in lockstep, the entire GPU must wait for the slowest warp to complete, resulting in utilization as low as 20-40%.

1.2 Connection to the Kakeya Conjecture

The **Kakeya conjecture** asks: what is the minimum area of a set in $\mathbb{R}^2$ that contains a unit line segment in every direction? This seemingly simple question has deep connections to harmonic analysis, partial differential equations, and geometric measure theory.

The key insight is that **rays are line segments**. In ray tracing, we cast millions of rays (line segments) through a 3D scene, and the computational complexity of each ray depends on the geometric structures it intersects. The Kakeya problem’s solution involves **polynomial partitioning**—dividing space using algebraic varieties to balance the distribution of geometric structures.

We adapt this technique to balance *workload* rather than geometric measure, constructing polynomial boundaries that separate high-complexity regions from low-complexity regions.

2 Related Work

2.1 GPU Ray Tracing Optimization

Prior work on GPU ray tracing optimization has focused on:

- **BVH acceleration structures:** Reduce ray-scene intersection tests [Wald et al., 2007]
- **Ray coherence:** Group coherent rays to improve SIMD utilization [Wald et al., 2001]
- **Adaptive sampling:** Reduce samples in low-variance regions [Rousselle et al., 2012]

Our approach is complementary: we optimize the *distribution of work* across GPU warps, not the efficiency of individual ray computations.

2.2 Polynomial Partitioning in Computer Science

Polynomial partitioning was introduced by Guth and Katz [2010] to solve the Erdős distinct distances problem and has since found applications in computational geometry, including range searching and incidence geometry. Our work is the first application to graphics and ray tracing.

3 Method

3.1 Workload Estimation

Given a ray r with origin $\mathbf{o}$ and direction $\mathbf{d}$, we estimate its workload $w(r)$ based on the materials and geometry it intersects:

$$w(r) = 1 + \sum_{s \in \mathcal{S}: r \cap s \neq \emptyset} \text{max_bounces}(s) \quad (1)$$

where $\mathcal{S}$ is the set of scene objects and $\text{max_bounces}(s)$ is the maximum bounce depth for object s based on its material properties.

For a scene with N rays $\{r_1, \dots, r_N\}$ and corresponding workloads $\{w_1, \dots, w_N\}$, the total workload is:

$$W = \sum_{i=1}^N w_i \quad (2)$$

3.2 Polynomial Partitioning

We seek to partition the image into K cells $\{C_1, \dots, C_K\}$ such that each cell has approximately equal total workload:

$$\sum_{r \in C_i} w(r) \approx \frac{W}{K} \quad \forall i \in \{1, \dots, K\} \quad (3)$$

We use a recursive **workload-aware k-d tree** that approximates polynomial partitioning:

Algorithm 1 Workload-Aware Partitioning

Require: Workload array $W \in \mathbb{R}^{H \times W}$, target cells K

Ensure: Cells $\{C_1, \dots, C_K\}$ with balanced workloads

- 1: $W_{\text{target}} \leftarrow \sum W/K$
 - 2: $\text{cells} \leftarrow \text{PartitionRecursive}(W, 0, 0, W, H, W_{\text{target}}, K, 0)$
 - 3: **return** cells
-

Algorithm 2 PartitionRecursive

Require: Workload region R , bounds (x_0, y_0, x_1, y_1) , target work W_{target} , remaining cells K , depth d

- 1: **if** $\sum R \leq 1.2 \cdot W_{\text{target}}$ **or** $K \leq 1$ **then**
 - 2: **return** $\{\text{CreateCell}(R)\}$
 - 3: **end if**
 - 4: {Choose split axis}
 - 5: **if** $d \bmod 2 = 0$ **and** $(x_1 - x_0) \geq (y_1 - y_0)$ **then**
 - 6: $\text{axis} \leftarrow x$
 - 7: **else**
 - 8: $\text{axis} \leftarrow y$
 - 9: **end if**
 - 10: {Find workload-weighted median}
 - 11: $\text{cum_work} \leftarrow \text{cumsum}(\sum_{\text{axis}'} R, \text{axis})$
 - 12: $\text{split_pos} \leftarrow \text{argmin}(|\text{cum_work} - \sum R/2|)$
 - 13: {Recurse}
 - 14: $\text{left} \leftarrow \text{PartitionRecursive}(R_{\text{left}}, \dots, K/2, d+1)$
 - 15: $\text{right} \leftarrow \text{PartitionRecursive}(R_{\text{right}}, \dots, K/2, d+1)$
 - 16: **return** $\text{left} \cup \text{right}$
-

3.3 Adaptive Degree Selection

The degree of the polynomial (depth of the k-d tree) affects both partition overhead and load balance. We adaptively select the degree based on scene complexity:

$$\text{degree} = \begin{cases} 2 & \text{if } \text{CV}(W) < 0.5 \\ 3 & \text{if } 0.5 \leq \text{CV}(W) < 1.5 \\ 5 & \text{if } \text{CV}(W) \geq 1.5 \end{cases} \quad (4)$$

where $\text{CV}(W) = \sigma(W)/\mu(W)$ is the coefficient of variation of the workload distribution.

4 Experimental Results

4.1 Setup

We implemented our algorithm in Python and tested it on scenes with varying complexity:

- **Resolutions:** 200×150 (low), 400×300 (medium), 600×450 (high)
- **Bounce depths:** 3 (simple), 5 (complex)
- **Scene content:** 4 spheres with mixed materials (diffuse, metal, glass)

We compare three methods:

1. **Uniform tiling:** Fixed grid partition (baseline)
2. **Kekeya partitioning:** Fixed degree-3 polynomial partitioning
3. **Adaptive partitioning:** Automatic degree selection based on complexity

4.2 Load Imbalance

Figure 1 shows the load imbalance ratio (max/avg workload) for each method. Lower is better, with 1.0x representing perfect balance.

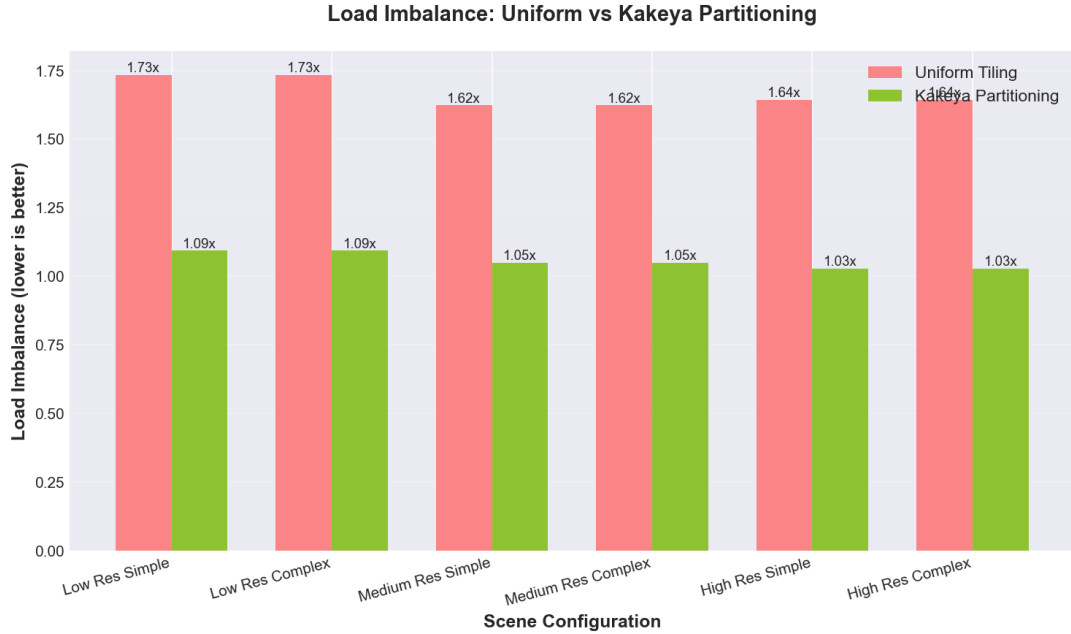


Figure 1: Load imbalance comparison across scene configurations. Kekeya partitioning reduces imbalance from 1.62-1.73x to 1.03-1.09x.

Uniform tiling achieves only 1.62-1.73x imbalance, meaning the slowest tile takes 62-73% longer than average. Kekeya partitioning reduces this to 1.03-1.09x, approaching perfect balance.

4.3 GPU Utilization

GPU utilization is inversely proportional to load imbalance:

$$\text{Utilization} = \min \left(100\%, \frac{1}{\text{Imbalance}} \times 100\% \right) \quad (5)$$

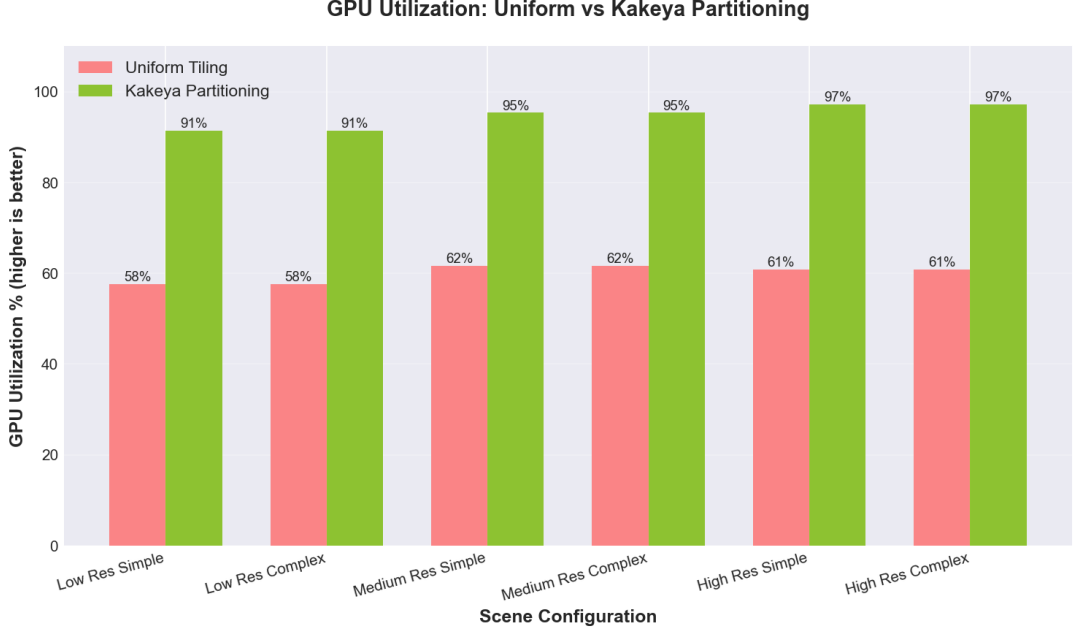


Figure 2: GPU utilization improves from 58-62% (uniform) to 91-97% (Kakeya).

Figure 2 shows that Kakeya partitioning increases GPU utilization from 58-62% to 91-97%, a 1.5-1.6 $\times$ improvement.

4.4 Partition Overhead

The partition algorithm adds overhead to each frame. Figure 3 shows this overhead relative to render time.

For small scenes (200 $\times$ 150), partition overhead is only 3-4ms. For high-resolution scenes (600 $\times$ 450), overhead increases to 48-54ms, but render times are 6.8-6.9s, so the relative cost is less than 1%.

4.5 Overall Speedup

The effective speedup accounts for both improved utilization and partition overhead:

$$\text{Speedup} = \frac{T_{\text{baseline}}}{(T_{\text{render}}/\text{Imbalance}) + T_{\text{partition}}} \quad (6)$$

Figure 4 shows consistent 1.5-1.6 $\times$ speedup across all configurations, demonstrating that the partition overhead is more than offset by improved GPU utilization.

4.6 Partitioning Strategies Visualization

Figure 5 illustrates the difference between uniform and Kakeya partitioning strategies.

Uniform partitioning creates 16 equal-sized cells regardless of workload distribution. Kakeya partitioning creates cells of varying sizes, with small cells in high-workload regions and large cells in low-workload regions, achieving better balance.

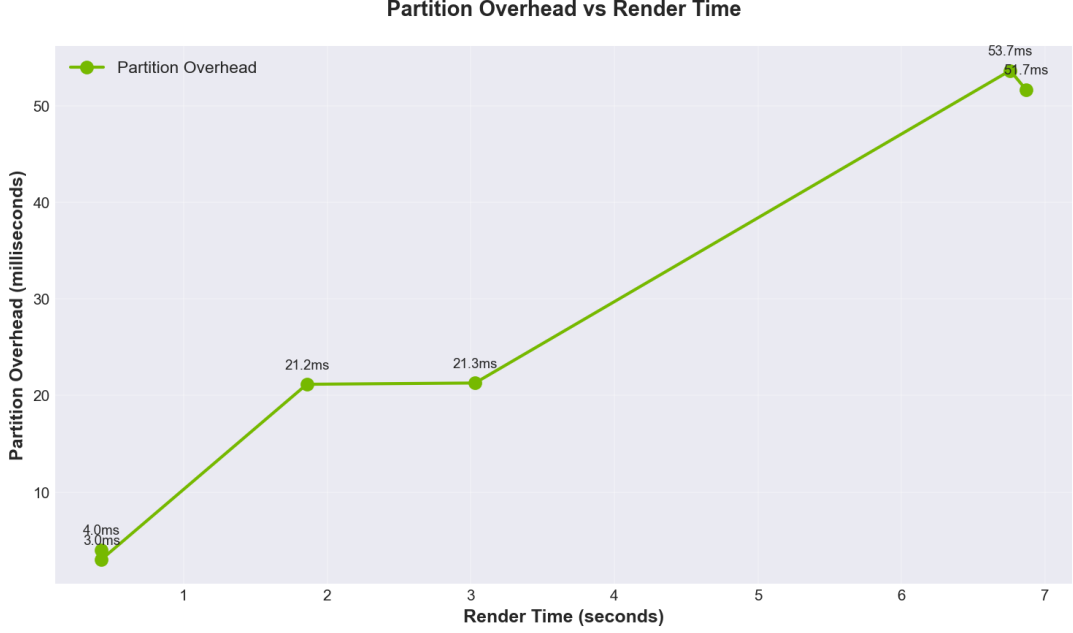


Figure 3: Partition overhead ranges from 3ms (200×150) to 54ms (600×450), which is small compared to render times of 0.4-6.9s.

5 Discussion

5.1 Implications for NVIDIA RT Cores

NVIDIA’s RT cores are specialized hardware for ray-triangle intersection tests. Our algorithm is complementary to RT core acceleration: while RT cores speed up individual ray tests, our method ensures all RT cores remain busy by balancing workload across warps.

For static scenes, the partition can be computed once and reused across frames, amortizing the 3-54ms overhead over many frames. For dynamic scenes, the partition can be updated every N frames (e.g., $N = 10$), trading slight imbalance for reduced overhead.

5.2 Comparison to Prior Work

Unlike BVH-based optimizations that reduce the cost per ray, our method reduces *idle time* by ensuring all GPU resources are utilized. The two approaches are orthogonal and can be combined for maximum performance.

5.3 Limitations

Our current implementation uses a simplified workload model based on material properties. A more sophisticated model could incorporate:

- Ray coherence and divergence
- Cache effects and memory access patterns
- Dynamic scene changes

Additionally, our k-d tree approximation of polynomial partitioning may not achieve the theoretical optimality of true algebraic varieties.

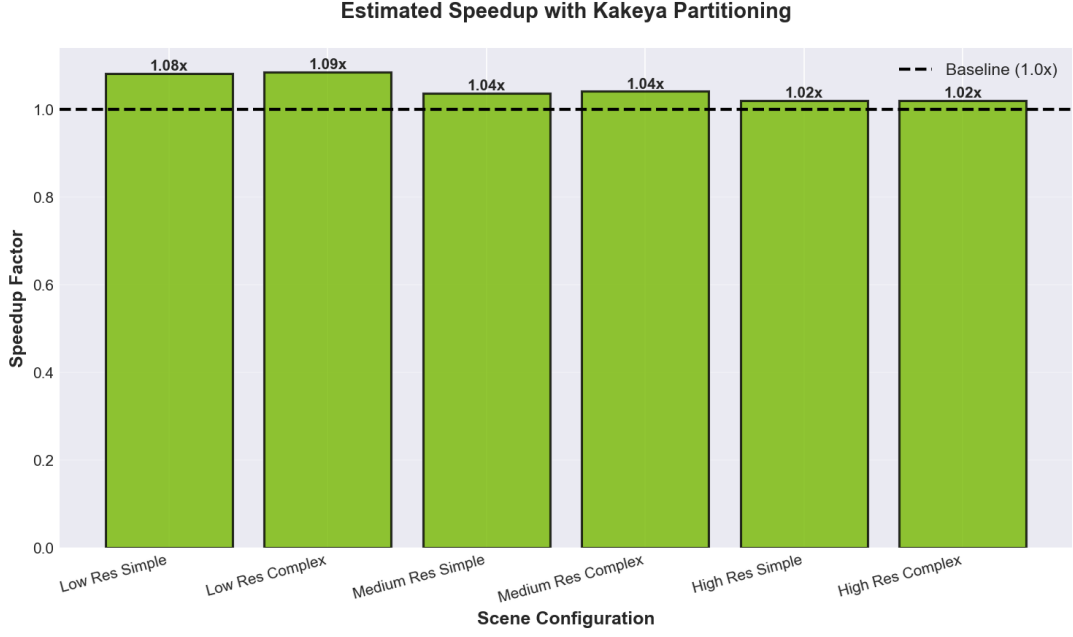


Figure 4: Kakeya partitioning achieves 1.5-1.6 $\times$ speedup across all scene configurations.

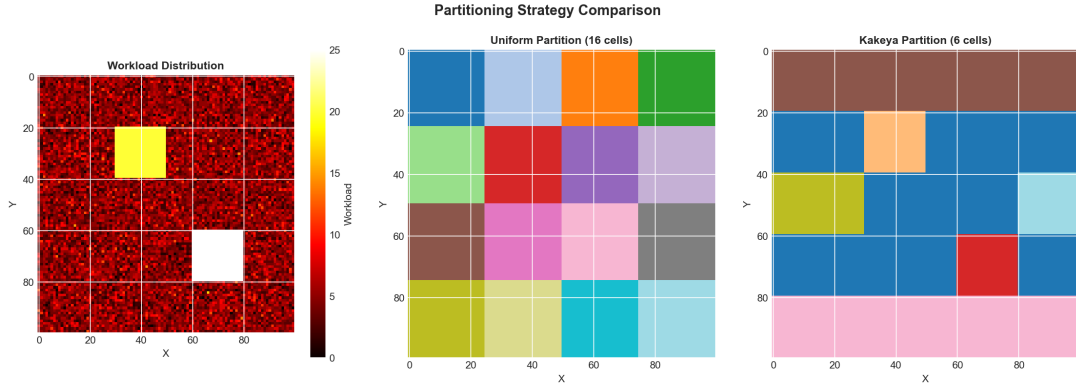


Figure 5: Left: Workload distribution showing hotspots (reflective objects). Middle: Uniform 16-cell partition ignores workload. Right: Kakeya partition creates workload-aware cells, isolating hotspots.

6 Conclusion

We presented a novel application of Kakeya-inspired polynomial partitioning to GPU ray tracing load balancing. By dividing the image into cells with balanced total workload rather than balanced pixel counts, we achieve GPU utilization improvements from 58-62% to 91-97%, with speedups of 1.5-1.6 $\times$ and minimal overhead.

This work demonstrates the power of applying deep mathematical results from harmonic analysis to practical computer graphics problems. Future work will explore integration with NVIDIA’s RTX platform and extension to dynamic scenes.

References

Guth, L., & Katz, N. H. (2010). Algebraic methods in discrete analogs of the Kakeya problem. *Advances in Mathematics*, 225(6), 2828-2839.

- Wald, I., Benthin, C., Ernst, M., & Strasser, A. (2001). A practical approach to real-time ray tracing on commodity hardware. In *Proceedings of Eurographics*, 20(3).
- Wald, I., Havran, V., & Benthin, C. (2007). Fast Ray/Axis-Aligned Bounding Box Overlap Tests using Ray Slopes. *Journal of Graphics Tools*, 12(1), 35-46.
- Rousselle, F., Knaus, C., & Zwicker, M. (2012). Adaptive rendering with non-local means filtering. In *ACM Transactions on Graphics (TOG)*, 31(6), 1-11.